

ApplyWizz DicePilot

Document 05 - Backend Schema

Data model, relationships, idempotency, and service boundaries for the V1 pilot

Product	ApplyWizz DicePilot
Version	V1.0 - Pilot Specification
Scope	V1 only: one internal test candidate, up to 20 Dice C2C + Easy Apply jobs, sequential processing
Date	20 August 2026

1. Schema Principles

- Reuse Supabase/PostgreSQL as the source of truth for business/application state.
- Do not duplicate the full ApplyWizz candidate profile into DicePilot; store candidate_id and retrieve current candidate facts through the existing route.
- Separate job identity from candidate-specific application state.
- Use database uniqueness and atomic claims to prevent duplicate application execution.
- Keep an append-only event history for debugging and auditability.
- Persist only references to browser artifacts/resumes where possible; avoid duplicating sensitive content.

2. Recommended Tables

Table	Purpose
dice_jobs	Normalized Dice job discovery record independent of candidate.
applications	Candidate-specific application queue and current status.
application_events	Append-only timeline of worker/browser/state events.
interventions	Unknown question or security action requiring human input.
browser_profiles	Mapping from candidate to persistent Dice browser profile metadata.
candidate_answer_overrides	Optional V1-safe storage for explicitly approved reusable answers; can be omitted initially if intervention answers stay one-off.

3. Table: dice_jobs

Column	Type	Constraints / meaning
id	uuid	PK, default gen_random_uuid()
dice_job_id	text	Required; unique normalized Dice identifier
canonical_url	text	Required; normalized Dice job URL
title	text	Required
company_name	text	Nullable only if Dice page genuinely omits it
location	text	Nullable
employment_type	text	Normalized Dice employment type
is_third_party	boolean	Structured Dice filter/evidence result
description	text	Full job description needed for C2C evidence and later analysis
c2c_status	text/enum	CONFIRMED, LIKELY, NOT_C2C, UNKNOWN
c2c_reason	text	Short evidence/explanation; avoid unbounded LLM transcript
is_easy_apply	boolean	Positive detector result

easy_apply_evidence	jsonb	Detector metadata/version/selectors or observed label
discovered_at	timestampz	Required
last_checked_at	timestampz	Eligibility freshness
raw_metadata	jsonb	Optional bounded structured source metadata

4. Table: applications

Column	Type	Constraints / meaning
id	uuid	PK
candidate_id	uuid	Required; references existing ApplyWizz candidate identity logically
dice_job_id	uuid	FK -> dice_jobs.id
status	text/enum	QUEUED, PROCESSING, NEEDS_INPUT, SUBMITTING, SUBMITTED, FAILED_RETRYABLE, FAILED
current_step	text	Browser/application state such as OPEN_JOB or ANSWER_QUESTIONS
priority	integer	Default 100 or defined pilot queue order
attempt_count	integer	Default 0
worker_id	text	Nullable; worker claiming row
lock_acquired_at	timestampz	For stale-claim recovery
resume_source_url	text	Optional reference used for this attempt; consider restricted access
error_code	text	Short machine-readable reason
error_message	text	Sanitized diagnostic summary
verification_evidence	jsonb	Positive success evidence metadata
queued_at	timestampz	Required
started_at	timestampz	Nullable
submitted_at	timestampz	Only after verified success
finished_at	timestampz	Terminal completion time
updated_at	timestampz	Required

Critical uniqueness

Create UNIQUE(candidate_id, dice_job_id). This is the primary database-level protection against unintentionally queueing/submitting the same Dice job twice for the same candidate.

5. Table: application_events

Column	Type	Meaning
id	bigserial/uuid	PK
application_id	uuid	FK -> applications.id
event_type	text	e.g. worker_claimed, browser_ready, job_opened, needs_input, submit_clicked, submission_verified
step	text	State machine step
message	text	Sanitized human-readable description
metadata	jsonb	Bounded technical metadata; no wholesale candidate payloads
created_at	timestampz	Required

6. Table: interventions

Column	Type	Meaning
id	uuid	PK
application_id	uuid	FK -> applications.id
type	text	UNKNOWN_QUESTION, AUTHENTICATION, SECURITY_ACTION, MISSING_CANDIDATE_FACT, OTHER
question_text	text	Nullable; exact visible question where applicable
options	jsonb	Nullable list of visible choices
status	text	OPEN, ANSWERED, CANCELLED
answer	jsonb/text	Approved answer; restrict access and avoid unnecessary sensitive values
answered_by	text/uuid	Operator/candidate provenance
created_at	timestampz	Required
resolved_at	timestampz	Nullable

7. Table: browser_profiles

Column	Type	Meaning
id	uuid	PK
candidate_id	uuid	Unique for Dice V1 candidate
platform	text	dice
profile_key	text	Opaque internal key/path token; do not expose filesystem path to UI
status	text	ACTIVE, NEEDS_LOGIN, NEEDS_ACTION, DISABLED
last_authenticated_at	timestamptz	Nullable
last_used_at	timestamptz	Nullable
created_at	timestamptz	Required
updated_at	timestamptz	Required

8. Candidate Answers - Optional V1 Table

If the pilot needs reusable operator-approved answers, use a small provenance-aware table rather than mutating the upstream candidate profile implicitly.

Column	Type	Meaning
id	uuid	PK
candidate_id	uuid	Required
question_key	text	Normalized reusable concept, e.g. willing_to_relocate
answer	jsonb	Approved value
source	text	candidate_api, operator_confirmed, candidate_confirmed
is_sensitive	boolean	Controls reuse/display policy
created_at	timestamptz	Required
updated_at	timestamptz	Required

9. Relationships

Relationship	Cardinality
dice_jobs -> applications	One job can have many candidate applications over final product lifetime; V1 has one candidate.
applications -> application_events	One-to-many append-only.

applications -> interventions	One-to-many.
candidate -> browser_profiles	One candidate to one Dice profile in V1; final product may support more platform profiles.
candidate -> candidate_answer_overrides	One-to-many optional reusable facts.

10. Indexes

Index	Reason
UNIQUE dice_jobs(dice_job_id)	Canonical Dice dedupe.
UNIQUE applications(candidate_id, dice_job_id)	Candidate/job idempotency.
INDEX applications(candidate_id, status, priority, queued_at)	Fast sequential queue claim.
INDEX application_events(application_id, created_at)	Fast timeline.
INDEX interventions(application_id, status)	Needs-input retrieval.
UNIQUE browser_profiles(candidate_id, platform)	One active Dice profile mapping per candidate.

11. Atomic Queue Claim

The worker must not select a QUEUED row and then update it in two race-prone independent operations when later scaling. Implement a transaction/RPC/locked update that atomically changes one eligible row to PROCESSING and records worker ownership. Even though V1 uses one candidate/worker, this design prevents a rewrite in V2.

12. Status Transition Rules

From	Allowed to
QUEUED	PROCESSING
PROCESSING	NEEDS_INPUT, SUBMITTING, FAILED_RETRYABLE, FAILED
NEEDS_INPUT	PROCESSING, FAILED
SUBMITTING	SUBMITTED, FAILED_RETRYABLE, FAILED
FAILED_RETRYABLE	QUEUED or PROCESSING through explicit retry policy
SUBMITTED	No further application execution
FAILED	No automatic retry unless manually reclassified

13. Supabase / Access Model

- Server-side worker uses a privileged server credential stored only in runtime secrets.
- Internal UI should use the least privilege needed to read pilot state and create controlled queue actions.
- Candidate browser profiles are not stored as raw cookies in normal application tables.

- Resume URLs and candidate identifiers should be treated as private application data.
- RLS/service boundaries should prevent unauthorized candidate/application visibility when the final product becomes multi-tenant; V1 schema should not block that future design.

14. Service/API Contracts

Endpoint/service	Contract
Existing ApplyWizz candidate route	GET candidate by candidate_id -> client + additional_information payload.
DicePilot eligible jobs	Internal API/query -> list c2c_status in allowed set and is_easy_apply=true.
Queue jobs	Create candidate-specific application rows; reject duplicates.
Application status	Read applications + latest events for dashboard.
Intervention answer	Resolve one OPEN intervention with provenance; application may resume according to state policy.

15. Data Retention / Logging Rules

- Do not copy the entire candidate API payload into logs or events.
- Store error context needed to debug selectors/states without exposing private candidate answers unnecessarily.
- Browser screenshots/traces should have restricted storage and retention because application pages may contain PII.
- Do not store passwords in Supabase application tables.
- Do not write service keys or browser auth state to source control.

16. Migration Order

1. Create/normalize shared Supabase client configuration using environment variables.
2. Create dice_jobs.
3. Create applications with candidate/job uniqueness.
4. Create application_events.
5. Create interventions.
6. Create browser_profiles.
7. Add indexes and status constraints.
8. Optionally add candidate_answer_overrides only when the V1 intervention workflow requires reusable answers.
9. Seed a controlled pilot candidate reference and test jobs without copying raw PII into fixtures.